



Applied Software Project Report

By

Karanam Siva

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements for the degree of Master of Science in Computer Science

August, 2026



Scaler Mentee Email ID : sivakaranamiiiitn@gmail.com

Thesis Supervisor : Naman Bhalla

Date of Submission : 12/08/2026

© The project report of Karanam Siva is approved, and it is acceptable in quality and form for publication electronically

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

.....

Project Guide / Supervisor

DECLARATION

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 16/10/2024 to 23/06/2025, is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Karanam Siva

A handwritten signature in black ink, appearing to read 'K. Siva', with a stylized flourish at the end.

Date: 12 August 2026

ACKNOWLEDGMENT

Completing this program alongside a full-time job took a lot of patience from the people around me, and I want to acknowledge that here. I am grateful to my family for their constant support and understanding through the long stretches this project and the coursework demanded. I would also like to thank my mentor, Naman Bhalla, and the Scaler Neovarsity instructors for the guidance through the project module classes that this capstone is built on, and my peers in the program for the discussions and shared references that helped along the way.

Table of Contents

List of Tables	8
List of Figures	9
Applied Software Project	10
Abstract	10
Project Description	11
Objectives	11
Relevance and Real-World Applications	11
Project Flow	12
System Architecture	12
Requirement Gathering	14
Functional Requirements	14
User Management	14
Product Catalog	14
Cart & Checkout	14
Order Management	14
Payment	14
Notifications	14
Non-Functional Requirements	15
Security	15
Performance	15
Scalability	15
Reliability	15
Maintainability	16

Observability	16
Users and Use Cases	17
Feature Set	18
Class Diagrams	19
user-service	19
product-service	19
order-service	19
payment-service	19
Database Schema Design	20
Feature Development Process	25
Request Flow	25
What manual end-to-end testing found	26
What was verified end to end	27
Deployment Flow	28
VPC and networking	28
EC2, Elastic Beanstalk, and Security Groups	28
RDS and the rest of the data tier	28
Observability and CI/CD in this setup	29
Technologies Used	30
Spring Boot / Spring Cloud	30
MySQL + Flyway	30
Elasticsearch	30
Apache Kafka	31
Spring Security / OAuth2	31

Razorpay	31
Docker / docker-compose	31
GitHub Actions	32
Maven multi-module reactor	32
JUnit 5, Mockito, and AssertJ	32
Lombok	32
Conclusion	33
Key Takeaways	33
Practical Applications	33
Limitations	33
References	34

List of Tables

Table No.	Title	Page No.
3.1	Feature set by service	18

List of Figures

Figure No.	Title	Page No.
2.1	System Architecture	13
3.1	Use Case Diagram	17
5.1	Database Schema (four independent MySQL schemas)	24
6.1	Checkout Sequence	26
7.1	AWS Deployment Architecture	29

Applied Software Project

1. Abstract

This project is a backend e-commerce platform built as five independent Spring Boot microservices behind an API gateway, covering the parts of an online store that matter most from a systems point of view, signing users in, browsing and searching a product catalog, managing a cart and placing orders, taking a payment, and telling the customer what happened. Each service owns its own database and can be built, tested, and deployed on its own. That independence is the main design decision behind the project, rather than one large application, the system is split along the natural boundaries of an e-commerce business (accounts, catalog, orders, payments, notifications), with a service registry and gateway tying the pieces together.

Authentication is handled by a dedicated user-service that issues JWTs for regular API traffic and separately runs an OAuth2 authorization server for machine-to-machine clients, so the project demonstrates both approaches rather than only describing one. Product search runs on Elasticsearch alongside a paginated, sortable MySQL-backed listing. Orders are created through direct service-to-service calls to product-service and payment-service, and payment outcomes come back asynchronously through Kafka, which also drives the order confirmation and payment receipt emails sent by notification-service. Payment integration uses Razorpay, including webhook signature verification and a scheduled job that reconciles any payment whose webhook never arrives.

The result is a working, automatically tested, containerized system that comes up locally with a single docker-compose command, and a practical case study in what microservice architecture actually costs and buys, more moving parts and network calls, in exchange for services that can be built, scaled, and deployed independently. In industry terms, this is the same pattern behind most production retail platforms, and the tradeoffs explored here, service boundaries, eventual consistency between order and payment status, and reconciling state after a missed webhook, are the same ones those systems have to handle.

2. Project Description

E-commerce backends look simple from the outside, a product list, a cart, a checkout button. Underneath, they sit on top of a set of distributed-systems problems that don't show up until the system has more than one moving part, who is this request actually from, is the payment real, did the confirmation email go out, and what happens to an order if the service handling payment goes down half way through. This project builds a backend for a generic online store to work through those problems directly, using the same architectural pattern production e-commerce systems use (independently deployable services coordinated through a gateway and a message broker) instead of a single Spring Boot application with everything in one process.

Objectives

The objectives for this project were:

- Build authentication covering both JWT-based API auth and a genuine OAuth2 authorization server, with password hashing, refresh token rotation, and CSRF protection where the flow actually needs it.
- Build a product catalog with pagination, sorting, and full-text search backed by Elasticsearch.
- Build a cart-to-order-to-payment flow spanning three services that reconciles asynchronously.
- Integrate a real payment gateway (Razorpay), including webhook handling and scheduled reconciliation for payments a webhook never confirms.
- Use Kafka to decouple order and payment events from the services that react to them.
- Containerize every service and orchestrate them with docker-compose.
- Write real automated tests (unit, slice, and context tests) rather than boilerplate contextLoads() tests.
- Expose health checks, metrics, and structured, correlated logging for observability.

Relevance and Real-World Applications

The specific problem this project solves (a generic online store) is less important than the fact that the same shape of system shows up anywhere a business takes money online: a subscription platform, a marketplace, a ticketing site, or an internal procurement tool all need some combination of accounts, a catalog of things to buy, a checkout flow, and a payment integration that can fail in ways the rest of the system has to tolerate gracefully. The specific choices here (JWT plus a separate OAuth2 path for machine clients, Elasticsearch for search once a catalog gets large, Kafka to keep a slow notification path from blocking checkout, database-per-service so one team's schema change can't break another team's queries) are the

same choices a small engineering team makes as their product grows past the point where a single application and a single database are enough. Retail, logistics, fintech, and ticketing platforms all eventually restructure around these same boundaries, which is why the pattern is worth learning on a project this size rather than only reading about it.

Project Flow

The project was built service by service rather than feature by feature. `service-registry` and `api-gateway` came first, since every other piece needs somewhere to register and something to route through. Next came `user-service`, since everything downstream depends on authentication, followed by `product-service`, then `order-service`, which depends on both `user-service` (for auth) and `product-service` (for pricing and stock), then `payment-service`, which `order-service` calls during checkout, and finally `notification-service`, since it depends on events that both `order-service` and `payment-service` produce. That build order matches the actual dependency graph between the services, rather than an arbitrary feature checklist.

System Architecture

Figure 2.1 shows the resulting architecture: a single API gateway in front of five Spring Boot services, all registered with a Eureka service registry so the gateway and the services can find each other by name instead of hardcoded addresses. `order-service` talks to `product-service` and `payment-service` directly over HTTP (Feign) for the parts of checkout that need an immediate answer, and Kafka carries the `order-created` and `payment-completed` events that drive asynchronous work, updating order status once a payment resolves, and sending confirmation and receipt emails.

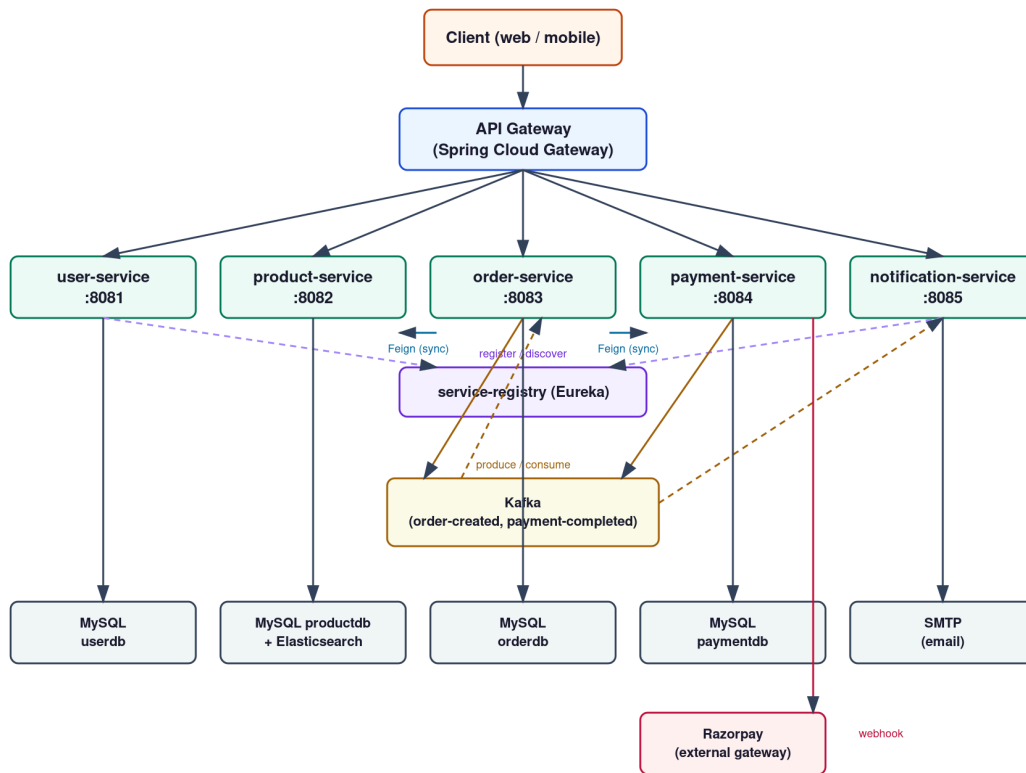


Figure 2.1: System Architecture

3. Requirement Gathering

Functional Requirements

User Management

- Registration with email and password (BCrypt-hashed).
- Login issuing a short-lived JWT access token plus an HttpOnly refresh cookie.
- Refresh token rotation on every use, revocation on logout.
- Role-based access (CUSTOMER / ADMIN).

Product Catalog

- Category and product CRUD, restricted to admins for writes.
- Paginated, sortable product listing, optionally filtered by category.
- Full-text product search via Elasticsearch.

Cart & Checkout

- Add and remove cart items, with live stock and price lookup from product-service.
- Checkout creates an order, initiates payment, and clears the cart.

Order Management

- Order history and order detail, scoped to the authenticated user.
- Order status updates automatically when payment succeeds or fails.

Payment

- Razorpay payment link creation for each order.
- Webhook-verified payment confirmation.
- Scheduled reconciliation for payments whose webhook never arrives.

Notifications

- Order confirmation email when an order is placed.
- Payment success or failure email when payment completes.
- Failed message processing is retried, then routed to a dead-letter topic.

Non-Functional Requirements

Security

- Passwords hashed with BCrypt, JWTs signed and validated with a shared secret across services, a separate OAuth2 authorization server (RSA-signed, client_credentials grant) for machine clients, CSRF protection on the cookie-based refresh endpoint, and role-based authorization on write endpoints. The two auth mechanisms are kept deliberately separate, a compromised OAuth2 client credential for a machine client can never be used to impersonate an end user, since the two token types are signed differently and validated by different code paths.

Performance

- Paginated endpoints wherever a list is returned, and an explicit JPA entity-graph override on product queries to avoid N+1 queries and LazyInitializationException when rendering category names outside the repository's transaction. The N+1 query problem this guards against is a common, easy-to-miss performance issue in any JPA-based service, since it doesn't show up in a quick manual test with one row of data, only under a realistic catalog size, which is exactly the kind of thing a schema diagram or a passing unit test won't surface either.

Scalability

- Each service is independently deployable and scalable (Eureka and Spring Cloud Gateway rather than hardcoded URLs), and Kafka decouples order and payment processing from notification-service so a slow mail server can never block checkout. In an AWS deployment this maps directly onto per-service Auto Scaling Groups (see Deployment Flow), product-service and order-service would typically need more instances than service-registry under real load, and Eureka-based discovery means that scaling decision doesn't require touching any other service's configuration.

Reliability

- Idempotent Kafka producers, a dead-letter topic for notification messages that fail repeatedly, scheduled payment reconciliation as a safety net for missed webhooks, and transactional order creation that rolls back cleanly if the payment call fails. The dead-letter topic in particular turned out to matter in practice, not just in theory, it is exactly what surfaced the Kafka type-header bug described in Feature Development Process, since messages that failed to deserialize showed up there instead of disappearing silently.

Maintainability

- Flyway-versioned schema migrations per service, automated tests (unit, MVC slice, repository slice, and context-load) across the reactor, and a shared common module for cross-cutting security and exception-handling code instead of duplicating it five times. Each service's Flyway migrations are numbered independently (V1, V2, ...), so adding a column to, say, payment-service's schema next month is a one-line SQL file in that service's own migration folder, with no coordination needed with the other three databases.

Observability

- Every service exposes /actuator/health, /actuator/metrics, and /actuator/prometheus, and structured logs carry a request id via a shared servlet filter so a single request can be traced across services. In practice this request id is what made the Feature Development Process bugs debuggable at all, grepping a service's logs for one request id shows every log line that request produced, in order, which is the difference between reading a specific failure and reading an entire log file.

Users and Use Cases

There are two actors, Customer (registers, logs in, browses and searches the catalog, manages a cart, checks out, and views order history) and Admin (manages categories and products). Figure 3.1 shows the use case diagram.

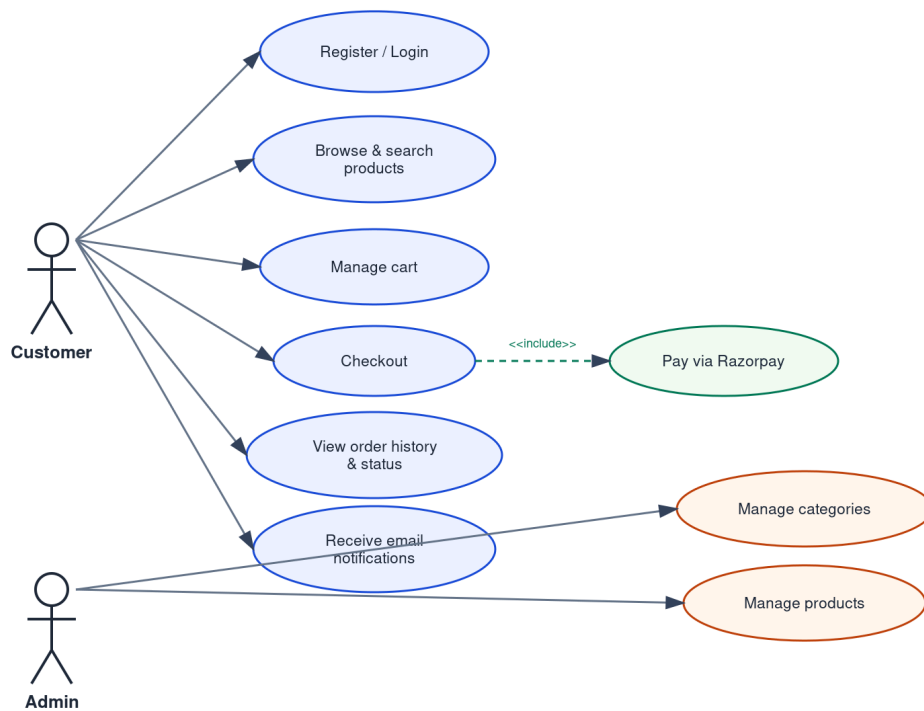


Figure 3.1: Use Case Diagram

Feature Set

Table 3.1 lists the implemented features by service, with the corresponding API endpoints.

Service	Feature	Endpoint(s)
user-service	Register / Login	POST /api/auth/register, POST /api/auth/login
user-service	Refresh / Logout	POST /api/auth/refresh, POST /api/auth/logout
user-service	Profile	GET /api/auth/me
user-service	OAuth2 client_credentials	POST /oauth2/token
product-service	Category management	GET/POST /api/categories
product-service	Product CRUD	GET/POST/PUT/DELETE /api/products
product-service	Full-text search	GET /api/search?q=
order-service	Cart management	GET/POST /api/cart, DELETE /api/cart/items/{id}
order-service	Checkout	POST /api/orders
order-service	Order history	GET /api/orders, GET /api/orders/{id}
payment-service	Payment creation	POST /api/payments (internal)
payment-service	Webhook	POST /api/payments/webhook
notification-service	Email notifications	Kafka consumer, no public endpoint

Table 3.1: Feature set by service

4. Class Diagrams

Each service's domain model is a small, independent set of JPA entities rather than one shared model, which is deliberate, user-service's User has nothing to do with order-service's idea of a customer beyond a shared numeric id. Figure 5.1 in the next section shows every table with its columns and keys. This section focuses instead on the relationships and the fetch-strategy decisions behind them, since those are the parts a schema diagram alone doesn't explain.

User-service

User ↔ Role is many-to-many, loaded eager, roles are small in number and checked on almost every authenticated request, so an extra query to lazy-load them on first access would just move the cost rather than remove it. User → RefreshToken is one-to-many, lazy: a user's refresh tokens are only ever looked up by hash during a refresh call, never by iterating the user's collection.

Product-service

Product → Category is many-to-one, lazy by mapping, but every repository method that actually returns a Product to a client (findAll, findByCategoryId, findById) overrides that with an @EntityGraph so the category is fetched in the same query. This is the fetch type vs. fetch mode distinction directly, the mapping stays lazy (correct default), and each query opts in to eager loading only where the response will use it.

Order-service

Cart → CartItem and Order → OrderItem are both one-to-many with cascade ALL and orphanRemoval, deleting a cart item by removing it from the parent's collection is enough, no separate repository call needed. CartItem and OrderItem intentionally do not hold a JPA relationship to Product. Instead, they store a plain productId plus a snapshot of the name and price at the time the item was added, since products live in a different service's database entirely.

Payment-service

Payment is a single entity with no relationships of its own. order_id and user_id are plain columns referencing rows in other services' databases, not foreign keys. That is a direct consequence of the database-per-service split, an order-service order and a payment-service payment can never be joined at the database level, only composed at the application layer.

5. Database Schema Design

Each service owns its own MySQL database, versioned with Flyway migrations, so a schema change in one service can never require a coordinated change in another. There is no database-level foreign key between services. Cross-service references (an order's `payment_id`, a payment's `order_id`) are plain columns resolved at the application layer via Feign calls and Kafka events. Figure 5.1 shows all four schemas, and the tables, keys, and cardinalities are listed below in the same textual format as the template's sample.

userdb (user-service)

Tables:

roles

- id
- name
- Primary Key(id)

users

- id
- name
- email
- password_hash
- enabled
- created_at
- Primary Key(id)

user_roles

- user_id
- role_id
- Primary Key(user_id, role_id)

refresh_tokens

- id
- token_hash
- user_id
- expires_at
- revoked
- created_at
- Primary Key(id)

Foreign Keys:

- user_roles(user_id) refers users(id)
- user_roles(role_id) refers roles(id)
- refresh_tokens(user_id) refers users(id)

Cardinality of Relations:

- Between users and roles -> m:m
- Between users and refresh_tokens -> 1:m

productdb (product-service)

Tables:

categories

- id
- name
- Primary Key(id)

products

- id
- name
- description
- price
- stock_quantity
- category_id
- created_at
- updated_at
- Primary Key(id)

Foreign Keys:

- products(category_id) refers categories(id)

Cardinality of Relations:

- Between products and categories -> m:1

orderdb (order-service)

Tables:

carts

- id
- user_id
- created_at
- Primary Key(id)

cart_items

- id
- cart_id
- product_id
- product_name
- unit_price
- quantity
- Primary Key(id)

orders

- id
- user_id
- status
- total_amount
- payment_id
- created_at
- updated_at
- Primary Key(id)

order_items

- id
- order_id
- product_id
- product_name
- unit_price
- quantity
- Primary Key(id)

Foreign Keys:

- cart_items(cart_id) refers carts(id)
- order_items(order_id) refers orders(id)

Cardinality of Relations:

- Between carts and cart_items -> 1:m
- Between orders and order_items -> 1:m

paymentdb (payment-service)

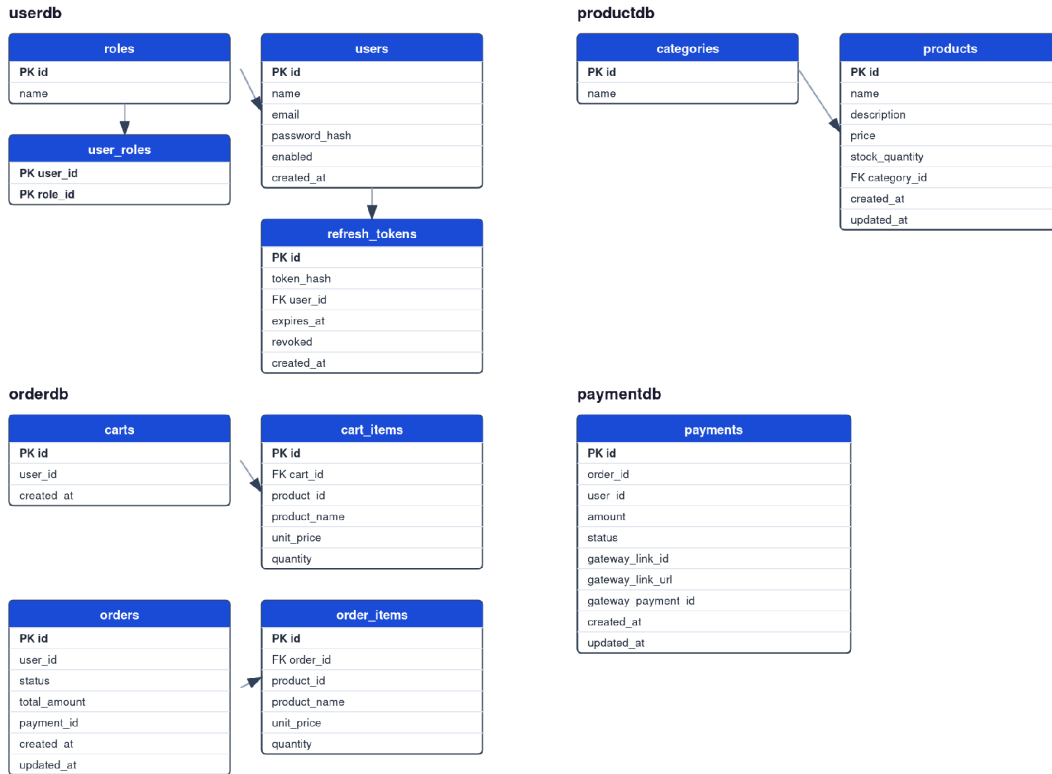
Tables:

payments

- id
- order_id
- user_id
- amount
- status
- gateway_link_id
- gateway_link_url
- gateway_payment_id
- created_at
- updated_at
- Primary Key(id)

Foreign Keys:

- none (order_id and user_id are logical references to orderdb/userdb, not physical foreign keys)



Dashed grey lines: foreign keys within a service's own database.
orders.payment_id and payments.order_id are logical references across services (no physical FK - different databases),
resolved at the application layer via Feign calls and Kafka events instead of a database join.

Figure 5.1: Database Schema (four independent MySQL schemas, one per service)

6. Feature Development Process

The feature covered here is checkout, adding an item to a cart, placing an order, and initiating payment. It was chosen because it is the one flow that touches every service in the system (order-service, product-service, payment-service, and indirectly notification-service through Kafka), so it exercises the architecture rather than a single service in isolation.

Request Flow

A client calls POST /api/orders with a bearer JWT. The request reaches the API gateway, which resolves order-service through Eureka and forwards it. OrderController extracts the user id and email from the JWT claims and calls OrderService.checkout(). That method loads the user's cart, and for each line item it already has a product name and price snapshotted from when the item was added to the cart (a synchronous Feign call to product-service's GET /api/products/{id} at add-to-cart time, not at checkout time). It builds an Order with OrderItems and saves it, then calls payment-service's POST /api/payments (again over Feign) with the order id, user id, and total amount. payment-service creates a Razorpay payment link and returns its id and URL. order-service stores that as the order's payment_id, marks the order PAYMENT_PENDING, clears the cart, and publishes an order-created event to Kafka. notification-service consumes that event and sends a confirmation email. All of this runs inside a single @Transactional method in OrderService, so if the payment-service call fails, the order insert and cart mutation both roll back rather than leaving a half-created order.

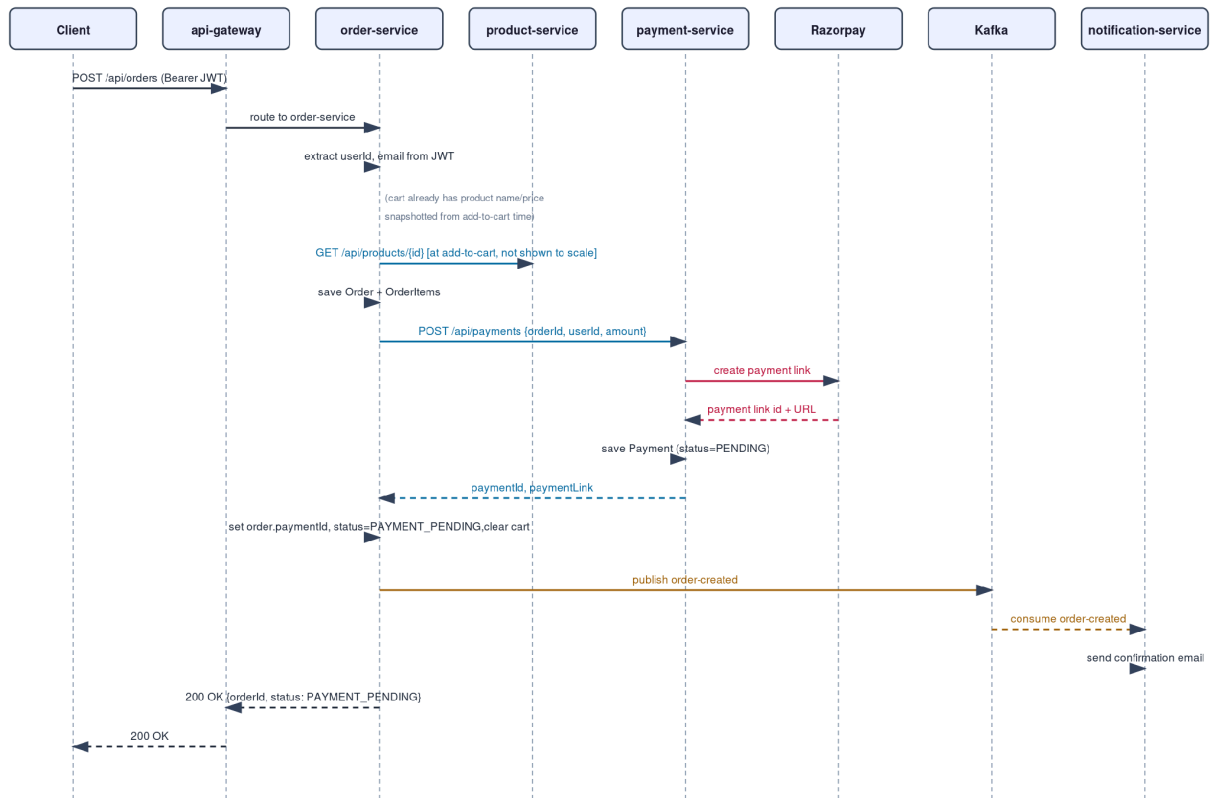


Figure 6.1: Checkout Sequence

What manual end-to-end testing found

Unit and slice tests passed throughout development, but bringing up the full docker-compose stack and calling the real endpoints through the gateway surfaced three bugs none of those tests caught, each now fixed with a regression test added specifically because it was missing.

- JWT algorithm mismatch: JwtService signed tokens with `signWith(key)` alone, which auto-selects HS256, HS384, or HS512 based on key length. Once the configured secret was long enough, it silently switched to HS384, while the shared resource-server decoder in common was hardcoded to HS256. Every authenticated request failed with a decode error. Fixed by specifying the algorithm explicitly on both sides, and added `JwtRoundTripTest`, which wires the real `JwtService` and the real `JwtDecoder` together instead of mocking either one.
- Missing entity graph on a single lookup, `ProductRepository.findAll` and `findByCategoryId` both had an `@EntityGraph` override to fetch category eagerly, but `findById` did not. With `spring.jpa.open-in-view` set to false, `GET /api/products/{id}` threw `LazyInitializationException` the moment the DTO mapper touched `product.getCategory()` outside the repository's transaction. Every existing test either mocked the repository or exercised a query

that already had the override, so nothing caught it until a real HTTP call did. Fixed by adding the same override to `findById`, with a new `@DataJpaTest` that clears the persistence context before asserting, so the test fails the same way production did.

- Kafka type header mismatch across services, Spring's `JsonSerializer` stamps the producer's own class name into a `__TypeId__` header by default. `order-service` and `notification-service` each keep a local copy of the same event records under their own packages rather than sharing a library module, so the consumer's deserializer tried to load a class that does not exist on its classpath, failed silently, and every message went straight to the dead-letter topic without ever reaching the `@KafkaListener`. There was no exception in application code to find, just an inbox that never received anything. Fixed with `spring.json.type.mapping` on the consumer side, verified with a test that drives the real `JsonDeserializer` against bytes shaped exactly like what the producer sends.

All three were config-and-wiring problems, not logic errors, which is exactly the class of bug that unit tests with mocked collaborators are structurally unable to catch: a mock resource-server decoder never disagrees with a mock JWT service about which algorithm was used, and a mocked Kafka consumer never has an opinion about what class name is in a message header. That is the practical argument for testing against the real running stack in addition to a unit test suite, not instead of one.

What was verified end to end

- Registration and login through the gateway, JWT issuance and validation across services.
- Adding a product to a cart, with a live Feign call to product-service for price and stock.
- Checkout creating a real order and a real Razorpay payment link (test-mode credentials, genuine API call, genuine payment link URL returned).
- The order-created Kafka event reaching notification-service and a real confirmation email arriving in a local mail catcher.

7. Deployment Flow

What is actually built and verified is the containerized side of this, a Dockerfile per service and a docker-compose stack that brings up all seven services plus MySQL, Elasticsearch, Kafka/Zookeeper, and a local mail catcher, tested end to end as described in the previous section. Standing up real AWS infrastructure was deliberately out of scope given the submission timeline (it costs money and time that a working, tested, containerized system already accounts for), and the submission itself asks for a repository link, a code archive, and this report, not a live URL. What follows is how the same containers would map onto AWS.

VPC and networking

One VPC spanning at least two availability zones for the pieces that support Multi-AZ (RDS, an Application Load Balancer). A public subnet holds only the ALB and the api-gateway instances, and nothing else is internet-facing. A private subnet holds service-registry, the five business services, RDS, Kafka, and the Elasticsearch cluster, reachable from the internet only indirectly, through the gateway.

EC2, Elastic Beanstalk, and Security Groups

Elastic Beanstalk's Docker platform is the practical choice here over hand-managed EC2 instances: every service already has a working Dockerfile and the same image that runs locally is what EB would deploy, one environment per service, without hand-building Auto Scaling Groups and load balancers for each one individually. service-registry runs as a single instance, since it is a coordination point rather than a scaling target, while the five business services scale independently based on their own load. Security Groups gate every hop, the ALB accepts inbound traffic from the internet and forwards only to api-gateway, api-gateway forwards only to the internal service Security Group, the internal Security Group accepts traffic only from the gateway and from other services in the same group (for the Feign calls and Eureka registration), and the data-tier Security Group (RDS, Kafka, Elasticsearch) accepts connections only from the internal service Security Group, never from the gateway or the internet directly.

RDS and the rest of the data tier

RDS for MySQL, Multi-AZ, hosting the four application schemas (userdb, productdb, orderdb, paymentdb) as separate schemas on one instance rather than four separate instances. This keeps cost down for a project this size, while still keeping each service's credentials and queries scoped

to its own schema, which is the isolation that actually matters (a bad query in one service cannot touch another service's rows). Amazon MSK would replace the docker-compose Kafka and Zookeeper containers with the same topics already implemented (order-created, payment-completed), and Amazon OpenSearch Service would replace the local Elasticsearch container backing product-service's search index.

Observability and CI/CD in this setup

Every service already exposes /actuator/health, /actuator/metrics, and /actuator/prometheus, and every log line carries a request id via the shared RequestIdFilter. In AWS, a CloudWatch Agent on each instance would scrape those endpoints and ship logs and metrics to CloudWatch, and CloudWatch Alarms would watch error rate and RDS load. The existing GitHub Actions workflow already builds and tests the full Maven reactor and builds all seven Docker images on every push. The only missing step for a real deployment is pushing those images to ECR and updating the corresponding Elastic Beanstalk environment, which needs AWS account access this project does not have.

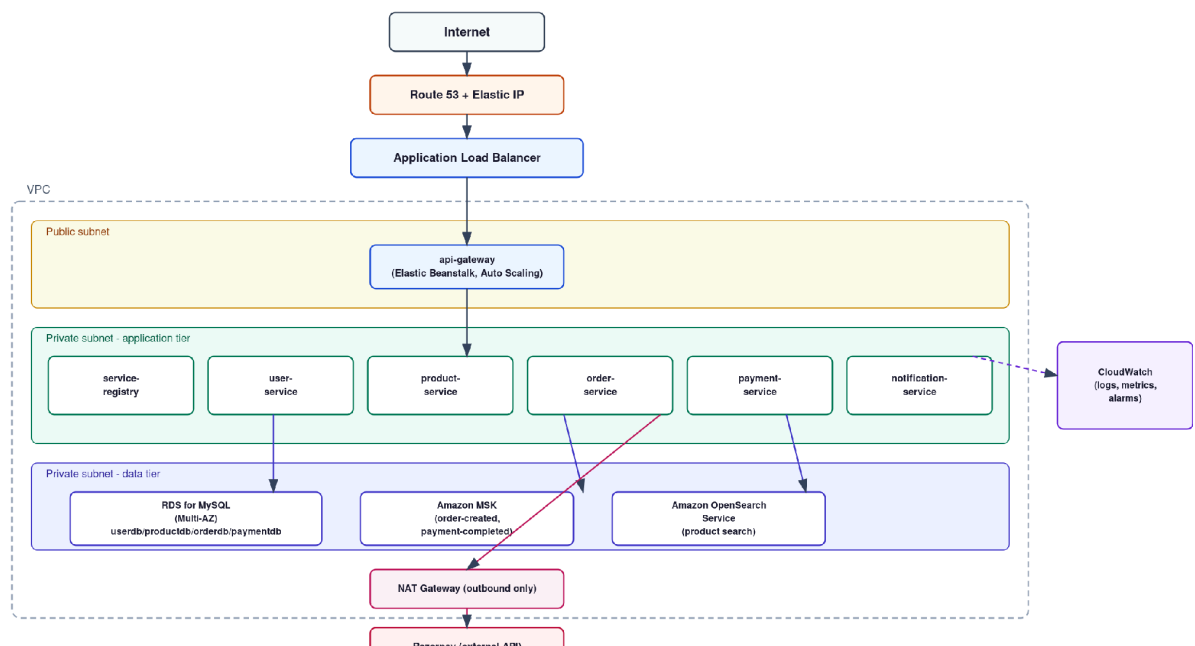


Figure 7.1: AWS Deployment Architecture

8. Technologies Used

This section covers the technologies behind the project and, for each one, what it is actually doing here and where it shows up in industry more broadly.

Spring Boot / Spring Cloud

The application framework for all seven services, plus Eureka (service discovery), Spring Cloud Gateway (routing), and OpenFeign (declarative HTTP clients between services). This combination is one of the most common ways Java shops build microservices in production, from fintech to retail, precisely because it turns service discovery and routing into configuration rather than code every team has to reinvent. Spring Boot 3.4 on Java 17 was chosen specifically because it is the current LTS combination the ecosystem is standardizing on, so the skills here transfer directly rather than being tied to a version already on its way out.

MySQL + Flyway

The relational store for all four business databases, with Flyway keeping schema changes versioned and reproducible instead of relying on Hibernate to auto-generate DDL. Versioned migrations are standard practice anywhere a schema needs to evolve safely across environments and teammates, not just here. Each service's migrations live under its own module (src/main/resources/db/migration), so userdb, productdb, orderdb, and paymentdb each have an independent version history, and a migration added to one service can never accidentally run against another service's schema.

Elasticsearch

Backs product-service's full-text search. Relational full-text search does not scale well past a certain catalog size or query complexity, which is why Elasticsearch is what most e-commerce and content platforms reach for once search becomes a first-class feature rather than a LIKE query. The product index maps name and description as text fields (tokenized, analyzed) and categoryName as a keyword field (exact match, used for future filtering), which is the standard split between "things users search" and "things users filter by" in a search index.

Apache Kafka

Carries order-created and payment-completed events between order-service, payment-service, and notification-service, so a slow or failing notification path can never block a checkout. This decoupling pattern (event-driven communication instead of every service calling every other service synchronously) is exactly how large-scale systems keep individual outages from cascading. Producers are configured with `acks=all` and `enable.idempotence=true`, so a network retry can never result in the same order-created or payment-completed event being delivered twice, which is a real concern once payment events are involved.

Spring Security / OAuth2

Two distinct mechanisms, deliberately, a custom JWT flow (login issues an access token plus an `HttpOnly` refresh cookie) for the API's own clients, and a separate Spring Authorization Server (RSA-signed, `client_credentials` grant) for machine-to-machine callers. JWTs are a token format, while OAuth2 is a protocol for issuing and delegating access, and this project uses OAuth2 to issue JWTs for one specific audience while issuing them directly, without OAuth2, for the other, a distinction that matters whenever a system needs to support both end users and third-party or internal service clients. The access token carries roles, email, and name as custom claims, which is what lets product-service and order-service authorize requests and personalize responses without an extra call back to user-service on every request.

Razorpay

The payment gateway integration, payment link creation, webhook signature verification, and refund-capable API access. Payment gateway integrations follow the same shape everywhere (create a charge or link, verify a webhook, reconcile anything the webhook missed), so the patterns here transfer directly to Stripe, PayPal, or any other provider. It was the natural choice for this project's context (UPI, net banking, and card support for the Indian market) over a US-centric gateway like Stripe, though the integration pattern here would look almost identical either way.

Docker / docker-compose

Every service is packaged as its own image, and docker-compose orchestrates all seven plus their infrastructure dependencies for local development and for the automated tests that run against a real stack. This is the same unit of deployment (a container) that Elastic Beanstalk, ECS, or

Kubernetes would run in production. Every Dockerfile is a multi-stage build (a JDK image compiles and packages the jar, a much smaller JRE image runs it), which is standard practice for keeping production images lean without giving up a full JDK during the build step.

GitHub Actions

Builds and tests the full Maven reactor and builds all seven Docker images on every push, catching build and test failures before they reach a deployment step. CI on every commit is close to universal in professional software teams at this point, for exactly that reason. Maven's dependency cache is keyed by pom.xml hash across runs, so a change to application code doesn't force a full re-download of the dependency tree on every push, only a change to a pom.xml does.

Maven multi-module reactor

The eight modules (common plus seven services) live in one Maven reactor build rather than eight separate repositories. A single `./mvnw test` at the root builds and tests every module in dependency order, and common is compiled once and reused rather than copy-pasted. The tradeoff is that all eight modules currently ship from one repository. Splitting them into independent repositories later would mean giving common its own release process instead of just depending on it as a reactor module.

JUnit 5, Mockito, and AssertJ

The testing stack across every module, JUnit 5 for test structure, Mockito for mocking collaborators in unit tests, and AssertJ for the fluent assertions used throughout (`assertThat(...).isEqualTo(...)` rather than JUnit's plain `assertEquals`). This combination is close to the default choice for testing in the Spring ecosystem, which is exactly why `spring-boot-starter-test` pulls in all three together.

Lombok

Generates getters, setters, and constructors for JPA entities and a few DTOs, which removes a meaningful amount of boilerplate from the entity classes in particular (a typical entity here has six or seven fields, which without Lombok would mean six or seven getter/setter pairs written by hand). It is excluded from the runnable jar via the Spring Boot Maven plugin configuration, since it is a compile-time-only annotation processor.

8. Conclusion

Key Takeaways

The biggest lesson from this project was not any single technology but how much of microservice architecture's real cost shows up only under integration testing, not unit testing. All three bugs described in the Feature Development Process section (a signing algorithm mismatch, a missing fetch-graph override, and a Kafka type-header mismatch across services) passed every unit and slice test and only surfaced once the full stack ran together through the actual gateway. That is a direct, hands-on argument for why service boundaries need to be tested as boundaries, not just as the sum of their parts.

Practical Applications

Every pattern here maps directly onto real e-commerce and fintech systems: JWT plus OAuth2 for authentication, event-driven order/payment reconciliation, Elasticsearch-backed search, and database-per-service isolation are not academic exercises, they are how platforms at meaningfully larger scale than this project actually work, which is exactly why the capstone brief asked for them.

Limitations

A few things are explicitly out of scope and worth naming rather than glossing over. There is no API-level authentication between services beyond the shared JWT secret. The internal POST `/api/payments` and GET `/api/internal/users/{id}` endpoints trust the network they run on rather than requiring a service credential, which is a reasonable tradeoff for a project this size but not for production. Payment reconciliation was verified against Razorpay's real API for payment-link creation, but the webhook path itself could not be exercised end to end since that requires a public URL Razorpay can call back to, which this environment does not have. No real AWS infrastructure was provisioned, for the cost and time reasons described under Deployment Flow. And Kubernetes, listed as a bonus topic in the capstone's own checklist, was not attempted, since Docker and docker-compose already cover the containerized deployment story this project needed to tell.

References

1. Spring Documentation Team, Spring Boot Reference Documentation, <https://docs.spring.io/spring-boot/index.html>
2. Spring Documentation Team, Spring Security Reference, <https://docs.spring.io/spring-security/reference/index.html>
3. Spring Documentation Team, Spring Authorization Server Reference, <https://docs.spring.io/spring-authorization-server/reference/index.html>
4. Spring Documentation Team, Spring Cloud Gateway Reference, <https://docs.spring.io/spring-cloud-gateway/reference/index.html>
5. Netflix / Spring Cloud Netflix, Eureka Wiki, <https://github.com/Netflix/eureka/wiki>
6. Apache Software Foundation, Apache Kafka Documentation, <https://kafka.apache.org/documentation/>
7. Spring Documentation Team, Spring for Apache Kafka Reference, <https://docs.spring.io/spring-kafka/reference/index.html>
8. Elastic NV, Elasticsearch Guide, <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>
9. CI/CD Best Practices - GitHub Actions, <https://docs.github.com/en/actions>
10. Red Hat / jjwt maintainers, Java JWT (jjwt) Documentation, <https://github.com/jwt/jjwt>
11. Flyway by Redgate, Flyway Documentation, <https://documentation.red-gate.com/fd>
12. Docker Inc., Docker Compose Documentation, <https://docs.docker.com/compose/>
13. Razorpay, Payment Links API and Webhooks Documentation, <https://razorpay.com/docs/>